

南开大学

汇编语言与逆向技术课程实验报告

实验五：peviewer



学院：密码与网络空间安全学院

专业：信息安全

学号：2412266

姓名：杨滢

班级：信息安全

1 实验时间

第 12 周星期一 9-10 节

2 实验目的

1. 熟悉 PE 文件结构;
2. 使用 Windows API 函数读取文件内容

3 实验环境

Windows 操作系统, MASM32 编译环境。

4 peviewer 程序的设计说明

4.1 程序功能

读取并解析 PE 文件的四个主要头部结构, 并将所有字段值以十六进制格式清晰显示。

4.2 文件操作

1. CreateFile: 以只读模式打开 PE 文件
2. SetFilePointer: 定位到文件开头
3. ReadFile: 读取文件内容到内存缓冲区
4. CloseHandle: 关闭文件句柄

4.3 PE 结构解析

4.3.1 IMAGE_DOS_HEADER

- e_magic: DOS 可执行文件签名
- e_lfanew: PE 文件头的偏移地址

实际地址 = 文件缓冲区基址 + 0

4.3.2 IMAGE_NT_HEADERS

- Signature: PE 文件格式签名

实际地址 = 文件缓冲区基址 + e_lfanew 值

4.3.3 IMAGE_FILE_HEADER

- NumberOfSections: 节区数量
- TimeDateStamp: 编译时间戳
- Characteristics: 文件特征标志

实际地址 = PE 头基址 + 文件头相对偏移

4.3.4 IMAGE_OPTIONAL_HEADER

- AddressOfEntryPoint: 程序入口点地址
- ImageBase: 默认加载基址
- SectionAlignment: 内存节区对齐
- FileAlignment: 文件节区对齐

实际地址 = PE 头基址 + 可选头相对偏移

5 peviewer 程序的控制流程图



6 peviewer.asm 的源代码和注释

```
1 .386
2 .model flat, stdcall
3 option casemap :none
4 ; 包含必要的头文件和库文件
5 include \masm32\include\windows.inc ; Windows API 常量、结构体和函数声明
6 include \masm32\include\masm32.inc ; MASM32 运行时库
7 include \masm32\macros\macros.asm ; MASM32 宏定义
8 include \masm32\include\kernel32.inc ; 内核函数
9 includelib \masm32\lib\masm32.lib ; MASM32 运行时库
10 includelib \masm32\lib\kernel32.lib ; 内核函数库
```

```

11 .data
12 que      BYTE "Please input a PE file: ",0          ; 文件输入提示
13 que1     BYTE "IMAGE_DOS_HEADER",0Ah,0Dh,0        ; DOS 头标识
14 que2     BYTE " e_magic: ",0                      ; DOS 签名字段
15 que3     BYTE " e_lfanew: ",0                     ; PE 头偏移字段
16 que4     BYTE "IMAGE_NT_HEADERS",0Ah,0Dh,0        ; NT 头标识
17 que5     BYTE " Signature: ",0                    ; PE 签名字段
18 que6     BYTE "IMAGE_FILE_HEADER",0Ah,0Dh,0       ; 文件头标识
19 que7     BYTE " NumberOfSections: ",0              ; 节数量字段
20 que8     BYTE " TimeDateStamp: ",0                 ; 时间戳字段
21 que9     BYTE " Characteristics: ",0              ; 文件特征字段
22 que10    BYTE "IMAGE_OPTIONAL_HEADER",0Ah,0Dh,0    ; 可选头标识
23 que11    BYTE " AddressOfEntryPoint: ",0           ; 入口点地址字段
24 que12    BYTE " ImageBase: ",0                     ; 映像基址字段
25 que13    BYTE " SectionAlignment: ",0              ; 内存对齐字段
26 que14    BYTE " FileAlignment: ",0                 ; 文件对齐字段
27 buf3     BYTE 4000 DUP(0)      ; 主文件内容缓冲区, 存储从文件读取的原始数据
28 buf4     BYTE 16 DUP(0)       ; 十六进制转换缓冲区, 用于存储转换后的十六进制字符串
29 buf5     BYTE 8 DUP(0)        ; 临时输出缓冲区, 用于格式化输出
30 file     BYTE 256 DUP(0)      ; 文件名输入缓冲区, 存储用户输入的文件名
31 hfile    DWORD 0              ; 文件句柄, 用于标识打开的文件
32 bytesRead DWORD 0             ; 实际读取字节数, 记录从文件读取的数据量
33 endl     BYTE 0Ah,0Dh,0       ; 换行符序列 (LF + CR), 用于控制台输出换行
34 temp     DWORD 0              ; 数据类型标志: 4表示WORD类型, 8表示DWORD类型
35 temp1    DWORD 0              ; 基址偏移量: 0表示DOS头基址, 非0表示PE头基址
36 pause_msg BYTE 0Ah,0Dh,"Press any key to exit...",0 ; 程序退出提示信息
37 .code
38 Output PROC
39     ; 保存被修改的寄存器, 遵循调用约定
40     push ebx
41     push ecx
42     push edx
43     push esi
44     mov esi, OFFSET buf3      ; ESI指向文件数据缓冲区起始位置
45     add esi, temp1            ; 加上结构体基址偏移 (DOS头或PE头)
46     add esi, edx              ; 加上字段在结构体内的具体偏移量
47     mov eax, DWORD PTR [esi]  ; 从计算出的地址读取4字节数据到EAX
48
49     invoke dw2hex, eax, addr buf4 ; 将EAX中的数值转换为8字符十六进制字符串
50                                     ; 结果存储在buf4中, 格式为"00000000"
51     mov ecx, temp              ; 获取数据类型标志
52     .if ecx == 8              ; 处理DWORD类型 (4字节, 8个十六进制字符)

```

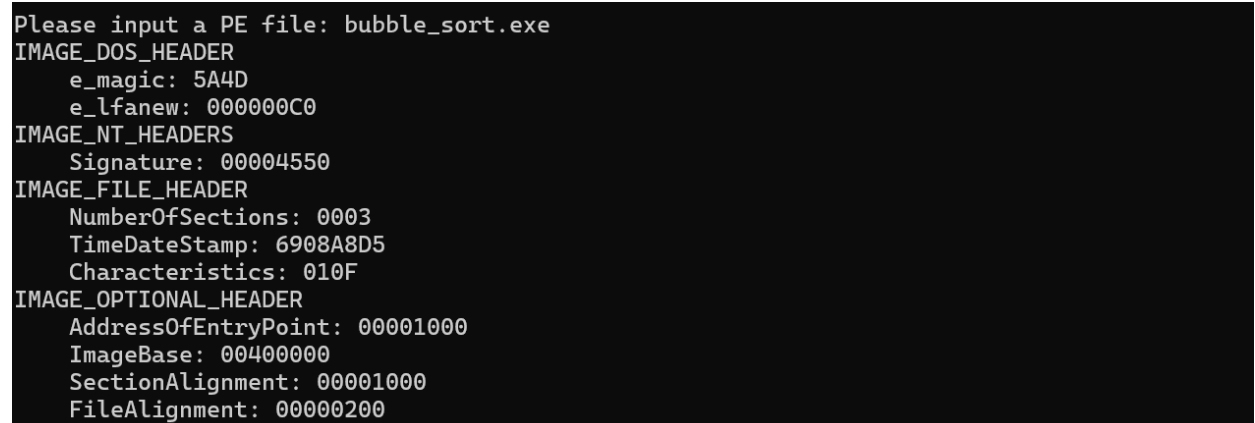
```
53     invoke StdOut, addr buf4 ; 直接输出完整的8字符十六进制字符串
54     .else                    ; 处理WORD类型 (2字节, 4个十六进制字符)
55         lea esi, buf4 + 4    ; 定位到buf4的第5个字符 (跳过前4个字符)
56                                ; 因为WORD类型只需要显示后4个十六进制字符
57         invoke StdOut, esi    ; 输出4字符十六进制字符串
58     .endif
59     invoke StdOut, addr endl  ; 输出换行符, 使显示结果更清晰
60     ; 恢复保存的寄存器, 保持堆栈平衡
61     pop esi
62     pop edx
63     pop ecx
64     pop ebx
65     ret
66 Output ENDP
67
68 start:
69     ; 显示文件名输入提示
70     invoke StdOut, addr que    ; 输出 "Please input a PE file: "
71     ; 获取用户输入的文件名
72     invoke StdIn, addr file, 256 ; 从标准输入读取文件名, 最大255字符+结束符
73     ; 检查用户是否输入了文件名
74     cmp byte ptr [file], 0     ; 检查文件名缓冲区的第一个字节
75     je exit_program            ; 如果为空, 直接退出程序
76     ; 打开指定的PE文件
77     invoke CreateFile, addr file, ; 文件名指针
78         GENERIC_READ, ; 访问模式: 只读
79         FILE_SHARE_READ, ; 共享模式: 允许其他进程读取
80         NULL, ; 安全属性: 默认
81         OPEN_EXISTING, ; 创建方式: 打开已存在文件
82         FILE_ATTRIBUTE_NORMAL, ; 文件属性: 普通文件
83         NULL ; 模板文件句柄: 无
84     mov hfile, eax ; 保存返回的文件句柄
85     ; 检查文件是否成功打开
86     cmp eax, INVALID_HANDLE_VALUE ; 比较返回句柄与无效句柄常量
87     je exit_program ; 如果打开失败, 跳转到退出处理
88     ; 设置文件指针到文件开头
89     invoke SetFilePointer, hfile, ; 文件句柄
90         0, ; 移动距离低32位
91         0, ; 移动距离高32位
92         FILE_BEGIN ; 移动基准: 文件开头
93     ; 读取文件内容到内存缓冲区
94     invoke ReadFile, hfile, ; 文件句柄
```

```
95         addr buf3,          ; 接收数据的缓冲区地址
96         4000,                ; 要读取的最大字节数
97         addr bytesRead,      ; 实际读取字节数变量的地址
98         NULL                 ; 重叠I/O结构: 不使用
99     ; PE结构解析与显示
100    invoke StdOut, addr que1   ; 输出"IMAGE_DOS_HEADER"
101    ; 显示e_magic字段
102    invoke StdOut, addr que2   ; 输出"    e_magic: "
103    mov edx, 0                ; 字段偏移: e_magic在DOS头中偏移0
104    mov temp1, 0              ; 基址选择: 使用DOS头基址 (缓冲区起始)
105    mov ecx, 4                ; 数据类型: WORD
106    mov temp, ecx
107    invoke Output              ; 调用输出过程显示该字段值
108    ; 显示e_lfanew字段 (PE头偏移)
109    invoke StdOut, addr que3   ; 输出"    e_lfanew: "
110    mov edx, 3Ch              ; 字段偏移: e_lfanew在DOS头中偏移0x3C
111    mov ecx, 8                ; 数据类型: DWORD
112    mov temp, ecx
113    invoke Output              ; 调用输出过程显示该字段值
114    ; 计算PE头在文件中的实际位置
115    mov esi, OFFSET buf3      ; 指向文件缓冲区起始
116    add esi, 3Ch              ; 加上e_lfanew字段的偏移量
117    mov eax, DWORD PTR [esi]  ; 读取e_lfanew的值
118    mov temp1, eax            ; 设置PE头基址偏移量
119    ; 显示NT头信息
120    invoke StdOut, addr que4   ; 输出"IMAGE_NT_HEADERS"
121    ; 显示Signature字段
122    invoke StdOut, addr que5   ; 输出"    Signature: "
123    mov edx, 0                ; 字段偏移: Signature在NT头中偏移0
124    mov ecx, 8                ; 数据类型: DWORD
125    mov temp, ecx
126    invoke Output              ; 调用输出过程显示该字段值
127    invoke StdOut, addr que6   ; 输出"IMAGE_FILE_HEADER"
128    ; 显示NumberOfSections字段
129    invoke StdOut, addr que7   ; 输出"    NumberOfSections: "
130    mov edx, 6                ; 字段偏移: 在文件头中偏移0x06
131    mov ecx, 4                ; 数据类型: WORD
132    mov temp, ecx
133    invoke Output              ; 调用输出过程显示该字段值
134    ; 显示TimeStamp字段
135    invoke StdOut, addr que8   ; 输出"    TimeDateStamp: "
136    mov edx, 8                ; 字段偏移: 在文件头中偏移0x08
```

```
137     mov ecx, 8                ; 数据类型: DWORD
138     mov temp, ecx
139     invoke Output              ; 调用输出过程显示该字段值
140     ; 显示Characteristics字段
141     invoke StdOut, addr que9   ; 输出"  Characteristics: "
142     mov edx, 16h              ; 字段偏移: 在文件头中偏移0x16
143     mov ecx, 4                ; 数据类型: WORD
144     mov temp, ecx
145     invoke Output              ; 调用输出过程显示该字段值
146     invoke StdOut, addr que10  ; 输出"IMAGE_OPTIONAL_HEADER"
147     ; 显示AddressOfEntryPoint字段
148     invoke StdOut, addr que11  ; 输出"  AddressOfEntryPoint: "
149     mov edx, 28h              ; 字段偏移: 在可选头中偏移0x28
150     mov ecx, 8                ; 数据类型: DWORD
151     mov temp, ecx
152     invoke Output              ; 调用输出过程显示该字段值
153     ; 显示ImageBase字段 (默认加载基址)
154     invoke StdOut, addr que12  ; 输出"  ImageBase: "
155     mov edx, 34h              ; 字段偏移: 在可选头中偏移0x34
156     mov ecx, 8                ; 数据类型: DWORD
157     mov temp, ecx
158     invoke Output              ; 调用输出过程显示该字段值
159     ; 显示SectionAlignment字段 (内存节区对齐)
160     invoke StdOut, addr que13  ; 输出"  SectionAlignment: "
161     mov edx, 38h              ; 字段偏移: 在可选头中偏移0x38
162     mov ecx, 8                ; 数据类型: DWORD
163     mov temp, ecx
164     invoke Output              ; 调用输出过程显示该字段值
165     ; 显示FileAlignment字段 (文件节区对齐)
166     invoke StdOut, addr que14  ; 输出"  FileAlignment: "
167     mov edx, 3Ch              ; 字段偏移: 在可选头中偏移0x3C
168     mov ecx, 8                ; 数据类型: DWORD
169     mov temp, ecx
170     invoke Output              ; 调用输出过程显示该字段值
171     invoke CloseHandle, hfile
172 exit_program:
173     ; 显示退出提示信息
174     invoke StdOut, addr pause_msg ; 输出"Press any key to exit..."
175     ; 等待用户按键
176     invoke StdIn, addr buf4, 2   ; 读取单个字符
177     ; 正常退出程序, 返回代码0
178     invoke ExitProcess, 0
```

179 end start

7 运行示例截图



```
Please input a PE file: bubble_sort.exe
IMAGE_DOS_HEADER
  e_magic: 5A4D
  e_lfanew: 000000C0
IMAGE_NT_HEADERS
  Signature: 00004550
IMAGE_FILE_HEADER
  NumberOfSections: 0003
  TimeDateStamp: 6908A8D5
  Characteristics: 010F
IMAGE_OPTIONAL_HEADER
  AddressOfEntryPoint: 00001000
  ImageBase: 00400000
  SectionAlignment: 00001000
  FileAlignment: 00000200
```

图 7.1: 运行结果